

Module 4 — Spring MVC

Spring MVC is the servlet-based web framework. The single most-asked MVC question is: **“Walk me through the complete request lifecycle through the DispatcherServlet.”** Master that flow and you cover most of this module.

4.1 DispatcherServlet & the Complete Request Lifecycle

Why Interviewers Ask This

It proves you understand the front-controller pattern and every extension point (handler mapping, argument resolvers, message converters, exception resolvers).

Core Concept

`DispatcherServlet` is the **front controller** — a single servlet that receives every request and orchestrates specialized components to produce a response.

Internal Working — the components

- **HandlerMapping** — maps URL+method → handler (`RequestMappingHandlerMapping`).
- **HandlerAdapter** — invokes the handler (`RequestMappingHandlerAdapter`).
- **HandlerMethodArgumentResolver** — binds params (`@RequestBody`, `@PathVariable`, `@RequestParam`, `@RequestHeader`, `Pageable`).
- **HttpMessageConverter** — serialize/deserialize body (Jackson for JSON).
- **HandlerInterceptor** — pre/post/afterCompletion hooks.
- **ViewResolver** — for view-based responses (Thymeleaf/JSP); skipped for `@ResponseBody`/REST.
- **HandlerExceptionResolver** — maps exceptions to responses (`@ExceptionHandler`, `ResponseStatusException`).

Lifecycle / Execution Flow

```
1. Request hits Servlet filters, then DispatcherServlet.doDispatch()
2. getHandler(): HandlerMapping -> HandlerExecutionChain (handler + interceptors)
3. interceptor.preHandle() (short-circuit if false)
4. HandlerAdapter: resolve arguments (ArgumentResolvers + MessageConverters)
5. Invoke controller method -> return value
6. Handle return: @ResponseBody -> MessageConverter writes body
   else -> ViewResolver -> render view
7. interceptor.postHandle()
8. render / write response
9. interceptor.afterCompletion()
   (any exception -> HandlerExceptionResolver -> error response)
```

ASCII — Full Flow

```

Client
| HTTP request
↓
[Servlet Filters] (CORS, security, correlation-id)
↓
DispatcherServlet → HandlerMapping → handler + interceptors
|
| → interceptor.preHandle()
| → HandlerAdapter → ArgumentResolvers → MessageConverter (JSON->obj)
|   ↓
|   @RestController method
|   ↓ returns object / view name
| → MessageConverter (obj->JSON) OR ViewResolver->View
| → interceptor.postHandle()
| → interceptor.afterCompletion()
↓
(exception anywhere) → HandlerExceptionResolver → @ExceptionHandler
↓
Response → [Filters unwind] → Client

```

Real Production Example

POST /orders with JSON: filters run → mapping finds `OrderController.create` → `@RequestBody` triggers Jackson to deserialize → `@Valid` validates → method runs → returns `Order` → Jackson serializes to JSON with `201 Created`.

Advantages / Trade-offs

Clean separation, huge extensibility. Thread-per-request (blocking) — for high I/O concurrency consider WebFlux. Lots of moving parts to learn.

Common Mistakes / Debugging

Missing `@ResponseBody`/`@RestController` → tries to resolve a view (404/500); wrong `Content-Type/Accept` → 415/406; `/actuator/mappings` to inspect routes; enable `logging.level.org.springframework.web=DEBUG`.

Interview Q / Follow-ups

- Walk through the DispatcherServlet request lifecycle.
- What are HandlerMapping / HandlerAdapter / MessageConverter?
- How does `@RequestBody` deserialize JSON? (*HttpMessageConverter / Jackson.*)
- Where do interceptors fire relative to the controller?
- How are exceptions turned into responses?

Hands-on Exercise

Register a `HandlerInterceptor` that logs request timing and add a `HandlerMethodArgumentResolver` for a custom `@CurrentUser` annotation.

Best Practices

Thin controllers (delegate to services); DTOs + validation; return `ResponseEntity` or `ProblemDetail`; keep blocking work off request threads when heavy.

4.2 Controllers, REST Controllers & Request Mapping

- `@Controller` returns view names; `@RestController` = `@Controller` + `@ResponseBody` (returns serialized body).
- `@RequestMapping` (class + method); shortcuts `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`.

- Params: `@PathVariable`, `@RequestParam`, `@RequestBody`, `@RequestHeader`, `@RequestPart` (multipart), `@ModelAttribute`.
- `produces` / `consumes` for content negotiation; `@ResponseStatus` for codes.

Interview Q

- `@RequestParam` vs `@PathVariable` vs `@RequestBody`.
- `@Controller` vs `@RestController`.
- How to return a specific status + headers? (`ResponseEntity`.)

4.3 Validation in MVC

`@Valid` / `@Validated` on `@RequestBody` / `@ModelAttribute` → binding errors → `MethodArgumentNotValidException` (body) / `BindException` (form) / `ConstraintViolationException` (method params on `@Validated` beans). Handle centrally (Module 3.6). See also Module 14.

4.4 Content Negotiation & Message Converters

Core Concept

Spring picks the response representation using the `Accept` header (and/or URL suffix/param).

`HttpMessageConverter`s translate between Java objects and wire formats: -

`MappingJackson2HttpMessageConverter` (JSON, default) - `MappingJackson2XmlHttpMessageConverter` / JAXB (XML) - `StringHttpMessageConverter`, `ByteArrayHttpMessageConverter`, form/multipart

ASCII

```
request Accept: application/json
-> ContentNegotiationManager picks JSON
-> RequestMappingHandlerAdapter selects Jackson converter
-> object serialized to JSON
```

Interview Q / Follow-ups

- How does content negotiation decide the format?
- How does `@RequestBody` / `@ResponseBody` use converters?
- Customize Jackson (dates, naming, null handling)? (`Jackson2ObjectMapperBuilderCustomizer`, `@JsonProperty`, `@JsonFormat`.)
- 406 vs 415 — which is which? (406 = can't produce `Accept`; 415 = can't consume `Content-Type`.)

Module 4 — One-Page Cheat Sheet

Component	Role
DispatcherServlet	front controller orchestrating everything
HandlerMapping	URL+method → handler
HandlerAdapter	invokes handler
ArgumentResolver	binds params (@RequestBody/@PathVariable/...)
HttpMessageConverter	JSON/XML ↔ objects (Jackson)
HandlerInterceptor	pre/post/afterCompletion
ViewResolver	view-based responses (non-REST)
HandlerExceptionResolver	exceptions → responses

Flow: filters → dispatcher → mapping → preHandle → adapter(args+converter) → controller → converter/view → postHandle → afterCompletion → response.

Module 4 — Top Interview Questions

1. Full DispatcherServlet request lifecycle (the classic).
2. HandlerMapping vs HandlerAdapter.
3. How `@RequestBody` / `@ResponseBody` work (message converters).
4. Content negotiation; 406 vs 415.
5. Filter vs interceptor placement.
6. `@Controller` vs `@RestController`.
7. Where and how validation errors surface.
8. How exceptions become HTTP responses.
9. `@RequestParam` vs `@PathVariable`.
10. Thread-per-request model implications.

Module 4 — Common Mistakes

- Forgetting `@ResponseBody` / `@RestController` (view resolution attempted).
- Wrong `consumes` / `produces` → 415/406.
- Business logic in controllers.
- Not handling `MethodArgumentNotValidException` globally.

Module 4 — Mock Interview

1. "Trace `POST /orders` with a JSON body end to end." → filters → dispatcher → mapping → preHandle → adapter → Jackson deserialize → `@Valid` → controller → service → Jackson serialize → 201 → afterCompletion.
2. "Client gets 406." → server can't produce the requested `Accept` type.
3. "How do I add a per-request correlation id visible in all logs?" → servlet filter sets MDC early (before dispatcher) and clears it after.
4. "How does Spring turn my `Order` object into JSON?" → `MappingJackson2HttpMessageConverter` selected by content negotiation.

5. “*When would you pick WebFlux over MVC?*” → very high concurrency, mostly I/O-bound, need backpressure/non-blocking clients.

Next → Module 5: Spring Data JPA & Hibernate.